

You are a senior full-stack engineer and AI architect building an MVP web app called **Candiq.AI** in Replit.

Goal:

Build a browser-based AI interview copilot that helps HR/recruiters run **subject-matter expert (SME) style interviews for ANY role** (technical or non-technical).

The app must:

- Capture audio from a live video call (tab audio + mic).
- Transcribe interviewer + candidate in real time using **OpenAI Whisper** (via OpenAI's speech-to-text API).
- Use the Job Description (JD) + SME notes + resume to:
 - Generate a bank of screening questions WITH rubrics.
 - Suggest SME-style questions during the interview.
 - Generate a structured, competency-based report and recommendation.

The AI is a **silent copilot**:

- Never joins the meeting as a bot.
- Never speaks to the candidate.
- Never makes final hiring decisions.

Non-goals (don't implement for MVP):

- Direct Zoom/Meet/Teams API integrations or bots.
- Video/facial analysis.
- TTS/voice output.
- ATS integrations.
- Multi-language support.
- Complex orgs/RBAC/billing.
- Mobile or browser extensions.

=====

1. CORE USER FLOW

=====

- 1) User logs in.
- 2) Creates a **Project** for a role.
- 3) Uploads JD (PDF/DOCX/text → plain text) and optionally SME notes (ideal profile, red flags, extra context).
- 4) System calls **Gemini** to:
 - Extract 5–12 key competency requirements based on the JD and the SME notes (if available)..
 - Generate 8–15 screening questions grouped by competency.
 - For each question, generate a rubric:

- typical_reasoning (what a good answer should cover),
- strong_signals[],
- weak_signals[],
- notes.

5) User lands on **Screening Question Setup**:

- Can edit question text, delete, add, reorder, and mark questions as mandatory/optional.
- Rubrics are shown (editing optional).
- On **Save**, final questions + rubrics are stored as the project's screening bank.

6) The system must support **regen** when SME context changes:

- JD + SME notes are editable.
- Button: "Regenerate from JD + SME notes".
- On click, call the same Gemini pipeline with updated context.
- Replace existing competencies + questions + rubrics after a confirmation dialog.
- Show regenerated questions in setup screen; use only after user saves again.

7) For each candidate, user creates an **Interview** under a project (candidate name, optional email + resume, resume → plain text).

8) User starts video call (Meet/Zoom/Teams) in another tab.

9) In Candiq.AI, user opens the **Interview Cockpit** and clicks

Connect Audio:

- Shares meeting tab with "Share tab audio".
- Grants mic permission.

10) During interview:

- Browser mixes tab audio + mic using Web Audio API and streams mixed audio to backend via WebSocket.
- Backend sends audio to **OpenAI Whisper** for streaming speech-to-text and receives partial/final transcripts.
- UI shows:
 - Live transcript with speaker labels in a small box.
 - A dominant box with the screening questions & rubric.
 - A suggestion box with up to 2 SME-style questions:
 - ONE follow-up (deepen on last answer).
 - ONE new screening question (prioritise mandatory/under-covered competencies).
 - Right panel with free-form notes + per-competency 1–5 ratings.
- User can mark suggestions or screening questions as **Asked** or **Dismissed**.

Asked suggestions update which questions/competencies are covered.

11) On **End Interview**:

- Backend calls **Gemini** with:
 - Role, JD + SME summaries.
 - Competency list.
 - Full/condensed transcript.
 - Screening questions asked + their rubrics.
 - Candidate answers mapped to question IDs.
 - Optional manual ratings/notes.
- Gemini returns JSON:
 - summary (2–3 paragraphs),
 - competencies[]: { name, score 1–5, reason },
 - recommendation: { decision: Hire/No-Hire/Hold, reason },
 - evidence[]: { point, competency, question_id | null }.
- Store as `report\_json` and render a PDF report.

=====

2. UX REQUIREMENTS

=====

- Desktop-only layout with 3 columns:
 - Left: role/candidate, timer, transcript (small)
 - Center: Suggestion Box (top) + screening questions and rubrics checklist. (bottom).
 - Right: notes + competency ratings.
- Show audio status (OK / no audio / reconnecting) and a recording banner.
- Show AI state near suggestions: “Listening...” vs “Thinking...”.
- On small/mobile screens, show: “Please use a desktop/laptop browser.”

=====

3. TECH + DATA REQUIREMENTS

=====

Frontend:

- React (or Next.js) + Tailwind CSS.
- State via Zustand/Context: auth, project, interview, transcript, suggestions, audio status.
- WebSocket or SSE for transcript and suggestion updates.

Audio + STT:

- `getDisplayMedia` for tab audio + `getUserMedia` for mic audio.
- Web Audio API to mix into one `MediaStream` → WebSocket to backend.
- Backend:
 - Sends mixed audio to **OpenAI Whisper** (OpenAI speech-to-text API),

preferably using its streaming interface.

- Receives partial/final transcripts and pushes them to the frontend.
- Stores transcript JSON with speaker, timestamp, text.

Backend:

- Node/Express or Next.js API routes.
- CRUD for users/projects/interviews.
- File upload + text extraction (JD, SME notes, resumes).
- Audio WebSocket endpoint for Whisper streaming.
- LLM orchestration with Gemini (for questions, suggestions, reports).

Database (Postgres):

- `users`: id, email, created_at.
- `projects`: id, user_id, title, jd_text, sme_notes_text, competency_rubric_json, screening_questions_json, created_at.
- `interviews`: id, project_id, candidate_name, candidate_email, resume_text, transcript_json, report_json, status, created_at.

LLM (Gemini) + STT (Whisper):

- Use **Gemini** for:
 - (1) Competency + question+rubric generation from JD + SME.
 - (2) Live suggestions (follow-up + new-topic) using recent transcript + question bank + coverage.
 - (3) Post-interview report using rubrics + answers.
- Use **OpenAI Whisper** ONLY for audio → text transcription.
- All API keys (GEMINI_API_KEY, OpenAI key) stored in Replit Secrets and used only on the backend.

Privacy:

- Consent checkbox before audio capture (“All participants consent...”).
- Persistent banner while capturing audio.
- Do not store raw audio; only transcripts + reports.
- Allow per-interview “Delete data” that removes transcript and report.

Success:

- A non-SME HR can:
 - Configure a role with JD + SME notes.
 - Get a good screening bank + rubrics.
 - Run a live interview with helpful suggestions.
 - Get a report that feels like an SME helped evaluate the candidate.